

《计算概论A》课程 程序设计部分习题课

董豪

北京大学 计算机学院

hao.dong@pku.edu.cn

北京大学

题目 - 魔王军队

来源：From Whf-2020 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

魔王需要带领他的军队从A城堡进攻B城堡。为了追踪他的兵力，他分发了很多观察之石。每颗观察之石的最大有效观察半径R个单位，并且必须被一个士兵携带（观察之石不能处于没有士兵的空中）。帮助魔王计算出为了追踪所有部队所需要的观察之石的最小数量。

关于输入

输入包含多个实例。每个测试实例第一行为两个整数：R，n。R（0<=R<=1000）是观察之石的有效半径，n（1<=n<=1000）为魔王军队的士兵数量。接下来的1行包含n个整数，分别是每个士兵的位置x_1,x_2,...,x_n（0<=x_i<=50000）。如果某个实例输入的R和n 均为-1，则结束。

关于输出

针对每个测试实例，输出一行整数表示所需的最小数量。

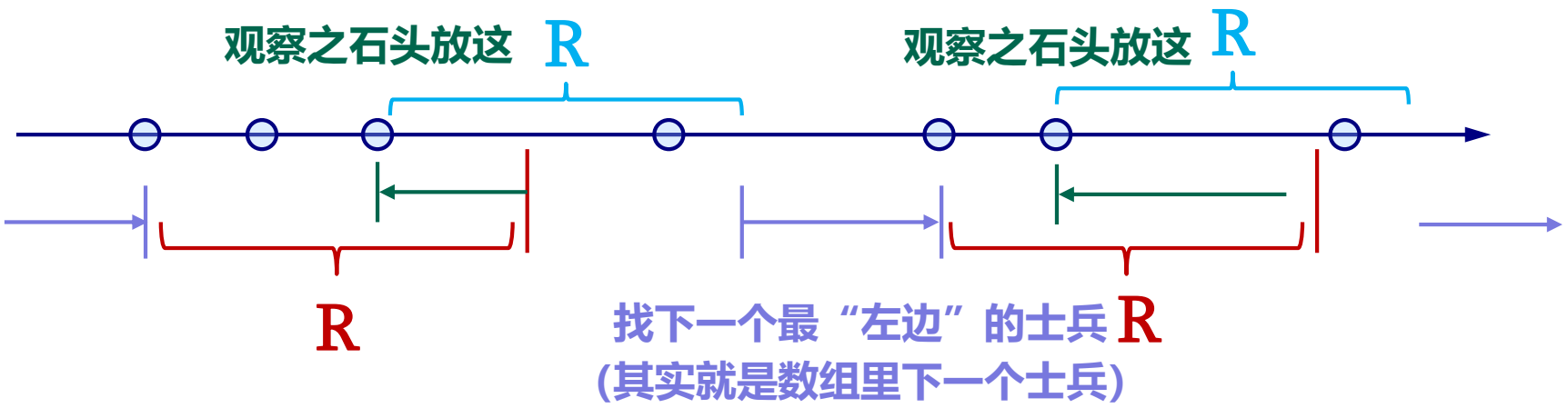
例子输入

```
0 3
10 20 20
10 7
70 30 1 7 15 20 50
-1 -1
```

例子输出

```
2
4
```




观察之石头放这 R
观察之石头放这 R
R
找下一个最“左边”的士兵 R
(其实就是数组里下一个士兵)



```cpp
#include <iostream>
#include <algorithm>
using namespace std;
int main(){
    int R = 0, n = 0;
    int soldiers[1000] = {0};
    while(cin>>R>>n){
        int stoneNum = 0;

        if(R == -1 && n == -1)
            break;

        for(int i = 0; i < n; i++)
            cin>>soldiers[i];

        sort(soldiers, soldiers+n);

        int stonePosition=0;

        while(stonePosition<=n-1){
            int leftR = 1, rightR = 1;
            while(soldiers[stonePosition + leftR] - soldiers[stonePosition] <= R)
                leftR++;

            stonePosition += leftR - 1;
            stoneNum++;

            while(soldiers[stonePosition + rightR] - soldiers[stonePosition] <= R)
                rightR++;

            stonePosition += rightR;
        }
        cout<<stoneNum<<endl;
    }
    return 0;
}
```

sort(begin, end, cmp)

- begin为指向待sort()的数组的第一个元素的指针
- end为指向待sort()的数组的最后一

```cpp
        for (int i = 0; i < n; i++)              // 外层循环
            for (int j = 0; j < n - i - 1; j++)    // 内层循环
            {
                if (soldiers[j] > soldiers[j+1] )
                {
                    int temp = soldiers[j];
                    soldiers[j] = soldiers[j+1];
                    soldiers[j+1] = temp;
                }
            }
//        sort(soldiers, soldiers+n);
```



关于sort函数

```
int main() {
    int arr[] = {3, 1, 4, 2, 5};
    int n = sizeof(arr) / sizeof(arr[0]); // 数组长度

    // 对arr[0]到arr[4]（共5个元素）排序（默认升序）
    sort(arr, arr + n);

    // 输出结果：1 2 3 4 5
    for (int num : arr) {
        cout << num << " ";
    }
    return 0;
}
```

北京大学



关于sort函数

```
int main() {
    vector<int> v = {5, 2, 9, 1, 5};

    // 对vector的所有元素排序（默认升序）
    sort(v.begin(), v.end());

    // 输出结果: 1 2 5 5 9
    for (int num : v) {
        cout << num << " ";
    }
    return 0;
}
```

北京大学



关于sort函数

■ 降顺排序

```
// 比较函数：若a > b，则a应排在b前面（降序）
bool cmp(int a, int b) {
    return a > b; // 核心逻辑：a "大于" b时，a在前
}

int main() {
    int arr[] = {3, 1, 4, 2};
    sort(arr, arr + 4, cmp); // 传入比较函数

    // 输出结果：4 3 2 1
    for (int num : arr) {
        cout << num << " ";
    }
    return 0;
}
```

```
int main() {
    vector<int> v = {3, 1, 4, 2};

    // 用lambda表达式定义降序规则
    sort(v.begin(), v.end(), [](int a, int b) {
        return a > b;
    });

    // 输出结果：4 3 2 1
    for (int num : v) {
        cout << num << " ";
    }
    return 0;
}
```

北京大学

题目 - 邮局选址

来源：医学部计算概论2020 (罗英伟) - yzh182-easy 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

在一个偏远的地区修建一个邮局，希望选择一个邮局地址，使得邮局到所有的这个地区所有村落的距离之和最小。 在这个问题中，我们假设所有村子位于一条直线上，每一个村子对应一个小于 100000 的整数x，代表村子在这个直线上的位置，邮局可以选择在直线上的任意一点，可以在一个村落中，也可以不位于村落中。

关于输入

输入由多组测试数据构成。
第一行为一个整数T，代表了测试数据的组数。
每组测试数据中 第一行是一个正整数n，输入数据中保证n大于等于2
第二行为n个数字，代表村落的坐标值。输入数据中保证村落的输入顺序按坐标从小到大排列。

关于输出

对于每组测试数据，每个测试数据输出一行，为一个整数，输出邮局到所有村落的距离之和。

例子输入

```
3
3
1 2 3
5
1 5 9 12 19
4
1 10 100 1000
```

例子输出

```
2
25
1089
```

```cpp
#include<iostream>
#include<cmath>
using namespace std;
int main(){
    int T = 0, n = 0, distance = 0, minDistance = 0;
    int village[100001] = {0};

    cin>>T;

    for (int i = 0; i < T; i++){

        cin>>n;

        for (int j = 0; j < n; j++)
            cin >> village[j];

        minDistance = 0x7fffffff;

        for (int j = 1; j < 100000; j++){

            distance = 0;

            for (int k = 0; k < n; k++)
                distance = distance + abs(j - village[k]);

             if (distance < minDistance)
                minDistance = distance;
        }
        cout<< minDistance <<endl;
    }
    return 0;
```

```cpp
#include <iostream>
#include <cmath>  // 需包含此头文件

int main() {
    double inf = INFINITY;          // 正无穷（double 类型）
    double neg_inf = -INFINITY;     // 负无穷（double 类型）

    std::cout << "正无穷: " << inf << std::endl;       // 输出 inf
    std::cout << "负无穷: " << neg_inf << std::endl;   // 输出 -inf

    return 0;
}
```

Time Out

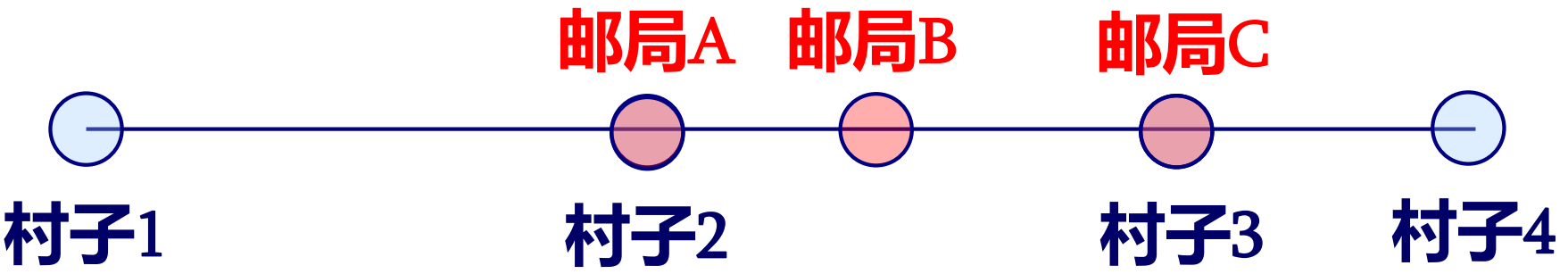
ProgramError: Case 0. Time out.



直接能想到的加速优化：不计算范围之外的地址



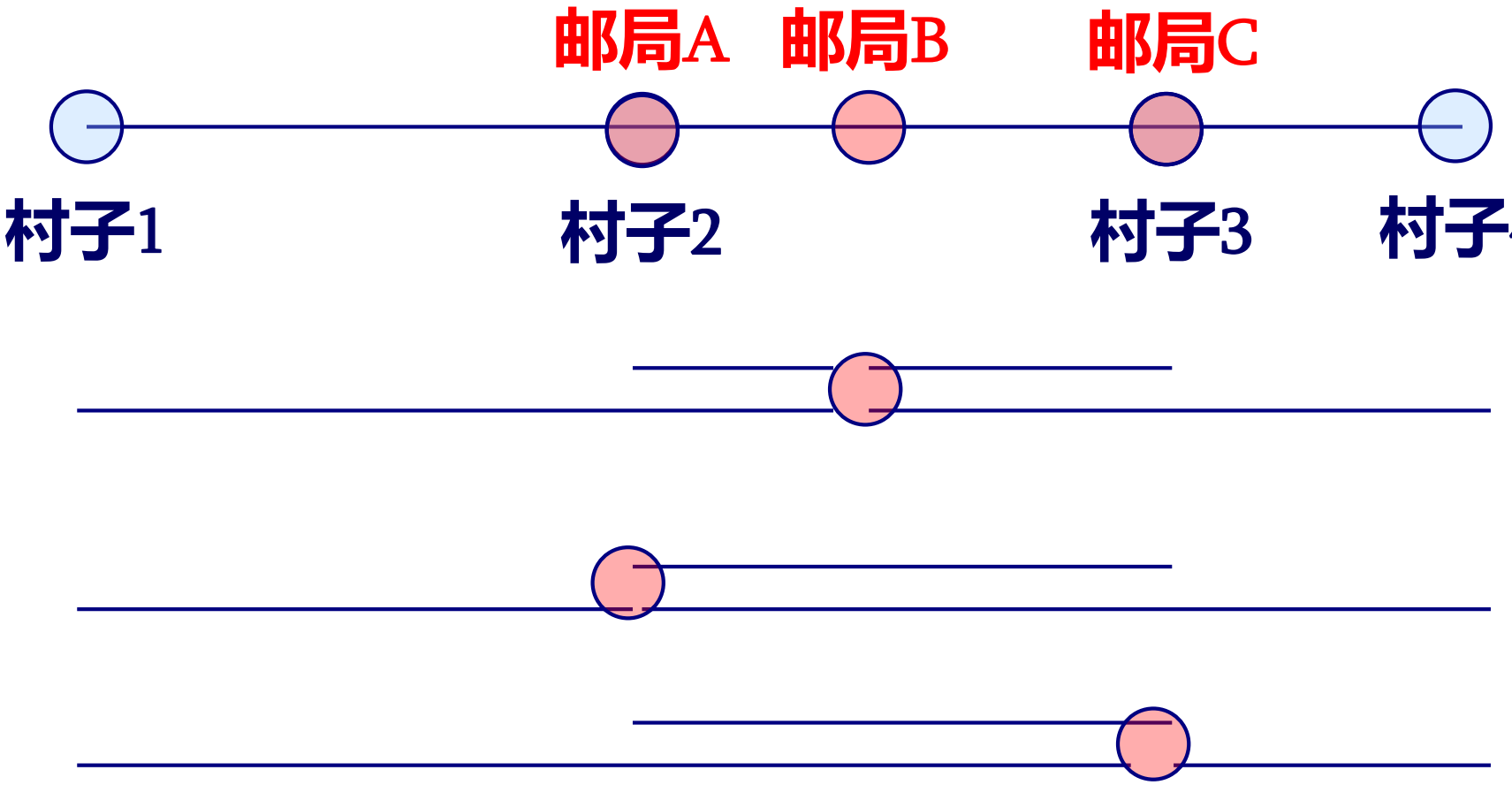
另外：



邮局A总距离 == 邮局B总距离 == 邮局C总距离

北京大学





邮局A总距离 == 邮局B总距离 == 邮局C总距离

北京大学

```cpp
#include<iostream>
#include<cmath>
using namespace std;
int main(){
    int T = 0, n = 0, distance = 0, minDistance = 0;
    int village[100001] = {0};
    cin>>T;
    for (int i = 0; i < T; i++){
        cin>>n;
        for (int j = 0; j < n; j++)
            cin >> village[j];

        minDistance = 0x7fffffff;

        for (int j = 1; j < n; j++){

            distance = 0;

            for (int k = 0; k < n; k++)
                distance = distance + abs(village[j] - village[k]);

            if (distance < minDistance)
                minDistance = distance;
        }
        cout<< minDistance <<endl;
    }
    return 0;
```

不计算村子范围之外的地址

只需要计算邮局在村子上的情况，
不需要计算所有可能的地址

x1≤x2≤...≤xn 时，当x取它们的中位数时求和有最小值

```
#include <iostream>
using namespace std;
/*
* 题意即为求|x-x1|+|x-x2|+...+|x-xn|的最小值
* 由几何直观:
* 1.当n为奇数时,x=x((n+1)/2)时上式最小
* 2.当n为偶数时,x位于xn/2和x(n+2)/2时最小
*/
int pos[100001];
int main()
{
        int T, n;
        cin >> T;
        while (T--)
        {
                cin >> n;
                for (int i = 1;i <= n;i++)
                        cin >> pos[i];
                int sum = 0;
                for (int i = 1;i <= n;i++)
                {
                        sum += abs(pos[i] - pos[(n + 1) / 2]);
                }
                cout << sum << endl;
        }
        return 0;
}
```

北京大学

题目 – 佩奇的勇气机

显示 编辑 发布 删除

来源	医学部计算概论–〈熊瑞勤〉–〈赵菁〉

描述

佩奇的期末考试马上就要开始了, 但是他没有足够的勇气报名参加, 为了帮助佩奇,她的小伙伴给她准备一台勇气获得机。初始的时候佩奇的勇气值是0, 勇气获得机有两个按钮:

1、N按钮: 如果当期拥有的勇气值为x, 按下之后勇气值将变为2*x+1,

2、G按钮: 如果当前拥有的勇气值为x, 按下之后勇气值将变为2*x+2,

勇气值过高也会膨胀,所以佩奇需要将自己的勇气值恰好变为n, 请你帮助她设计一个勇气获得机的按键方案使妞妞的勇气值恰好变为n。

关于输入

输入包括一行, 包括一个正整数n(1 <= n <= 100000), 表示佩奇最后需要的勇气值。

关于输出

输出一行字符串, 每个字符表示该次佩奇选择按动的按钮,'N'表示该次按动N按钮,'G'表示该次按动G按钮。

例子输入

```
20
```

例子输出

```
NGNG
```

```cpp
#include<iostream>
using namespace std;
int main() {
    int n;
    cin >> n;    char a[100000];
    int num = 0;
    while (n!=0)
    {
        if (n % 2 == 0)
        {
            n = (n - 2) / 2;
            a[num++] = 'G';
        }
        else
        {
            n = (n - 1) / 2;
            a[num++] = 'N';
        }
    }
    for (int i = num-1; i >= 0; i--)
    {
        cout << a[i];
    }
}
```

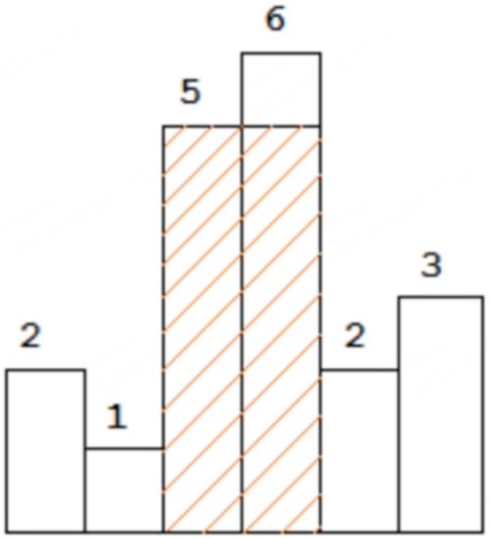
题目 - 柱状图上的最大矩形

显示 编辑 发布 删除

来源	曲直

描述

给定n个非负整数，代表柱状图上每个柱的高度（宽度均为1），求这个柱状图中最大的矩形面积。例如对于输入"2 1 5 6 2 3"，最大面积为10（见下图）



例子输入

6

2 1 5 6 2 3

例子输出

10

关于输入

第一行是一个整数n，代表有多少个柱形，n小于等于20000
第二行有n个整数，依次为每个柱形的高度

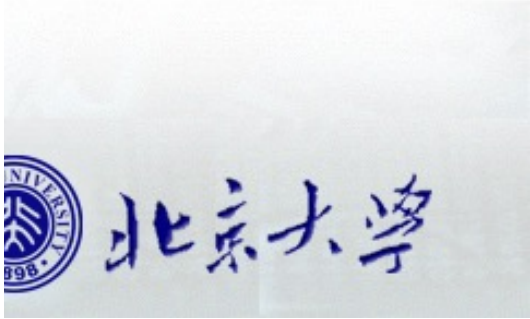
关于输出

一个数字，即这个柱状图中最大的矩形面积，面积最大不超过int的表示范围

```cpp
#include <iostream>
using namespace std;
int main()
{
    int n;
    cin >> n;
    int height[n], leftlower[n], rightlower[n]; // 柱形图高度 左边比它低的第一个矩形编号 右边比它低的第一个矩形编号
    for (int i = 0; i < n; i++)
        cin >> height[i];
    for (int i = 0; i < n ; i++) // 寻找每个矩形左右比它低的一个矩形的编号
    {
        leftlower[i] = -1;
        rightlower[i] = n;
        for (int j = i - 1; j >= 0; j--)
            if (height[j] < height[i])
            {
                leftlower[i] = j;
                break;
            }
        for (int j = i + 1; j < n; j++)
            if (height[j] < height[i])
            {
                rightlower[i] = j;
                break;
            }
    }
    int maxsquare = 0, square;
    for (int i = 0; i < n; i++) // 计算矩形面积
    {
        square = height[i] * (rightlower[i] - leftlower[i] - 1);
        maxsquare = (square > maxsquare) ? square : maxsquare;
    }
    cout << maxsquare << endl;
    return 0;
}
```



```cpp
#include <iostream>
using namespace std;
int main()
{
    int n;
    cin>>n;
    int height[20000]={0};
    int i=0;
    for (i=0;i<n;i++)
    {
        cin>>height[i];
    }//高度存储完毕
    int S[20000]={0};
    i=0;
    int j=0;
    for (i=0;i<n;i++)
    {
        for (j=i+1;j<n;j++)
        {
            if (height[j]>=height[i])
            {
                S[i]=S[i]+height[i];
            }
            else break;
        }
        for (j=i;j>=0;j--)
        {
            if (height[j]>=height[i])
            {
                S[i]=S[i]+height[i];
            }
            else break;
        }
    }//所有面积存储完毕
    int max=0;
    i=0;
    for (i=0;i<n;i++)
    {
        if (S[i]>max) max=S[i];
    }
    cout<<max<<endl;
    return 0;
}
```

题目 - 习题(15-10) 试剂稀释

来源： 教师页面

[提交程序] [IDE] [提交记录]

题目描述

一种药剂可以被稀释成不同的浓度供病人使用，且只能稀释不能增加浓度；又已知医院规定同一瓶药剂只能给某个病人以及排在他后面的若干人使用。现为了能最大限度利用每一瓶药剂(不考虑每一瓶容量)，在给出的一个病人用药浓度序列(病人的顺序不能改变)中找出能同时使用一瓶药剂的最多人数。

关于输入

有两行，第一行是一个整数n，为病人的人数，假设不超过100；第二行为一个浮点数（double）序列，为每个病人的用药浓度，浮点数之间用一个空格隔开。

关于输出

输出一行，该行包含一个整数，为所求的最大人数。

例子输入

```
6
0.7 0.9 0.6 0.8 0.8 0.4
```

例子输出

0.9 0.8 0.8 0.4可以按顺序用1瓶

```
4
```

```cpp
#include<iostream>
using namespace std;
int main()
{
    int n = 0; //病人的人数
    double concentration[101] = {0};
    int patientsNum[101] = {0};
    cin>>n;
    for (int i = 0; i < n; i++){
        cin>> concentration[i];
        patientsNum[i] = 1;
    }
    //对不同浓度的药剂i，检查有多少位病人j可以使用
    for(int i = 0; i < n; i++){
        for(int j = i+1; j < n; j++)
            if (concentration[j] <= concentration[i]
                                        && patientsNum[i] + 1 > patientsNum[j])
            {
                patientsNum[j] = patientsNum[i] + 1;
            }
    }

    int maxNum = 0;
    for (int i = 0; i < n; i++)
        if(patientsNum[i] > maxNum)
            maxNum = patientsNum[i];

    cout<< maxNum <<endl;

    return 0;
}
```



谢　谢！

北京大学